

APP DISTRIBUTION USING GITHUB ACTIONS AND FIREBASE

A general setup guide for automatically building and distributing an Android app to testers with a CI/CD pipeline.

Table of Contents

1. Version History
 2. Introduction
 3. How It Works
 4. Prerequisites
 5. Enable Firebase App Distribution & Add Testers
 6. Authentication — Create a Service Account
 7. Generate an Android Release Keystore
 8. Get your App ID
 9. Configure GitHub Repository Secrets
 10. Add a Gradle Release Signing Config
 11. Create the GitHub Actions Workflow
 12. Run the Pipeline
 13. Verify Build Release
 14. Test Build Release
 15. Mobile Application Installation Guide (Android)
 16. Troubleshooting
 17. Appendix — Secrets Reference
-

Version History

Version	Date	Author	Details
1.0	2026-08-23	cjdelacruz267@gmail.com	Initial guide

Introduction

This document explains how to set up a **CI/CD pipeline with GitHub Actions** that builds a **signed Android release APK** on every push and uploads it to **Firebase App Distribution**, so your testers can install the app

over-the-air — no USB cable or manual steps.

It authenticates using a **Firebase service account** (the modern method). The older `firebase login:ci` `FIREBASE_TOKEN` approach is deprecated and is **not** used here.

Throughout this guide, replace the placeholders with your own values:

Placeholder	Meaning
<PROJECT_ID>	your Firebase project id
<com.your.app>	your Android application id / package name
<OWNER>/<REPO>	your GitHub repository
<ANDROID_APP_ID>	your Firebase Android App ID (see Section 4)
<TESTER_GROUP>	the App Distribution tester group name (e.g. <code>testers</code>)
<ALIAS>	your keystore key alias

Official docs: [Firebase App Distribution](#) · [GitHub Actions](#)

How It Works

On every push to your main branch (or a manual trigger), the workflow:

1. checks out the repo and sets up the build tools (JDK, Android SDK, and Node if your app has a web/JS build such as Capacitor/Ionic/React Native web);
2. restores build-time secrets (signing keystore, `google-services.json`);
3. builds a **signed release APK** with Gradle; and
4. uploads it to **Firebase App Distribution**, which emails your testers.

```
push to main → GitHub Actions → build signed APK → Firebase App Distribution → testers
```

Prerequisites

1. A **Firebase project** (the **Blaze** plan is recommended; App Distribution itself is free).
2. An **Android app registered** in that Firebase project (provides `google-services.json` and the Android App ID).
3. A **GitHub repository** containing your app source, including the **Android project** (`android/` folder) so CI can build it.
4. **Owner/Editor** access to the underlying Google Cloud project (to manage IAM and service accounts).
5. For local builds: the Android SDK, a matching **JDK**, and (if applicable) Node.js.

Commit your `android/` folder. Some frameworks (e.g. Capacitor) git-ignore the native `android/` project by default. If your app customizes the native project (permissions, keys, plugins), commit the folder so CI can build it. Keep build outputs, `local.properties`, the **keystore**, and `google-services.json` git-ignored and inject the latter two from secrets.

1. Enable Firebase App Distribution & Add Testers

1. In the [Firebase Console](#), open your project → **Run** → **App Distribution** → **Get started**.
2. Open the **Testers & Groups** tab → create a group (e.g. `<TESTER_GROUP>`).
3. Add tester email addresses (any Google account) to the group.

2. Authentication — Create a Service Account

1. Firebase Console → ⚙ **Project settings** → **Service accounts** → **Generate new private key**; download the JSON file.
2. **Do not commit this file.** Store its **contents** in a GitHub secret named `FIREBASE_SERVICE_ACCOUNT` (Section 5).
3. In [Google Cloud Console](#) → **IAM**, grant the service account:
 - **Firebase App Distribution Admin** (to upload builds)
 - *(optional)* **Firebase Hosting Admin** — only if the same pipeline also deploys a website.

3. Generate an Android Release Keystore

Release APKs must be signed with a stable key so updates install over the previous version. Generate the keystore **once**:

```
keytool -genkeypair -v -keystore release.jks -alias <ALIAS> \
-keyalg RSA -keysize 2048 -validity 10000
```

You will set a **store password**, fill identity fields, and set a **key password** (press Enter to reuse the store password). Record the passwords and the alias.

Back up `release.jks` securely. If you lose it you cannot ship updates to the same app. Never commit it — add `*.jks` and `*.keystore` to `.gitignore`.

Encode the keystore (and `google-services.json`) to base64 so they can be stored as secrets.

PowerShell (pipes straight to the clipboard to avoid encoding issues):

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("release.jks")) | Set-Clipboard  
[Convert]::ToBase64String([IO.File]::ReadAllBytes("android/app/google-services.json")) | Set-Clipboard
```

macOS/Linux:

```
base64 -w0 release.jks  
base64 -w0 android/app/google-services.json
```

4. Get your App ID

App Distribution identifies the app by its **Android App ID** (different from the *web* App ID used by the front-end SDK). Find it via:

- Firebase Console → **Project settings** → **General** → your Android app → **App ID**, or
- the `mobilesdk_app_id` field in `android/app/google-services.json`.

It looks like: `1:1234567890:android:abcdef0123456789`.

5. Configure GitHub Repository Secrets

In GitHub: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Add:

Secret	Value
<code>FIREBASE_ANDROID_APP_ID</code>	your Android App ID (Section 4)
<code>FIREBASE_SERVICE_ACCOUNT</code>	the full service-account JSON (Section 2)
<code>GOOGLE_SERVICES_JSON_BASE64</code>	base64 of <code>google-services.json</code> (Section 3)
<code>ANDROID_KEYSTORE_BASE64</code>	base64 of <code>release.jks</code> (Section 3)
<code>ANDROID_KEYSTORE_PASSWORD</code>	keystore store password
<code>ANDROID_KEY_ALIAS</code>	your key alias (<code><ALIAS></code>)
<code>ANDROID_KEY_PASSWORD</code>	key password (same as store password if you pressed Enter)

Add any other app configuration your build needs (API keys, Firebase web config, etc.) as additional secrets and inject them at build time. Never hard-code secrets in the repo.

6. Add a Gradle Release Signing Config

In your app module's `build.gradle` (usually `android/app/build.gradle`), read the signing material from environment variables so **CI can inject it** and **local debug builds are unaffected**:

```
def ksFile = System.getenv('ANDROID_KEYSTORE_FILE')
def hasSigning = ksFile != null && !ksFile.isEmpty() && file(ksFile).exists()

android {
    defaultConfig {
        // Use the CI run number so each build is a distinct release.
        versionCode System.getenv('ANDROID_VERSION_CODE') ? System.getenv('ANDROID_VERSION_CODE') : 1
    }
    signingConfigs {
        if (hasSigning) {
            release {
                storeFile file(ksFile)
                storePassword System.getenv('ANDROID_KEYSTORE_PASSWORD')
                keyAlias System.getenv('ANDROID_KEY_ALIAS')
                keyPassword System.getenv('ANDROID_KEY_PASSWORD')
            }
        }
    }
    buildTypes {
        release {
            if (hasSigning) { signingConfig signingConfigs.release }
        }
    }
}
```

7. Create the GitHub Actions Workflow

Create `.github/workflows/distribute-android.yml`. Adjust paths to your project structure (the example includes optional Node/Capacitor steps — remove them for a pure-native app):

```

name: Distribute Android

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  distribute:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Optional: only if your app has a JS/web build (Capacitor/Ionic/React Native web)
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21          # match your app's required JDK

      - uses: android-actions/setup-android@v3

      # Optional web build (Capacitor example)
      - run: npm ci
      - run: npm run build
      - run: npx cap sync android

      - name: Restore google-services.json
        env:
          DATA: ${ secrets.GOOGLERVICES_JSON_BASE64 }
        run: |
          mkdir -p android/app
          echo "$DATA" | base64 -d > android/app/google-services.json

      - name: Decode keystore
        env:
          KS: ${ secrets.ANDROID_KEYSTORE_BASE64 }
        run: echo "$KS" | base64 -d > "$RUNNER_TEMP/release.jks"

      - name: Build signed release APK
        working-directory: android
        env:
          ANDROID_KEYSTORE_FILE: ${ runner.temp }/release.jks
          ANDROID_KEYSTORE_PASSWORD: ${ secrets.ANDROID_KEYSTORE_PASSWORD }
          ANDROID_KEY_ALIAS: ${ secrets.ANDROID_KEY_ALIAS }
          ANDROID_KEY_PASSWORD: ${ secrets.ANDROID_KEY_PASSWORD }
          ANDROID_VERSION_CODE: ${ github.run_number }
        run: |
          chmod +x ./gradlew
          ./gradlew assembleRelease

      - name: Upload to Firebase App Distribution
        uses: wzieba/Firebase-Distribution-Github-Action@v1
        with:

```

```
appId: ${ secrets.FIREBASE_ANDROID_APP_ID }
serviceCredentialsFileContent: ${ secrets.FIREBASE_SERVICE_ACCOUNT }
groups: <TESTER_GROUP>
file: android/app/build/outputs/apk/release/app-release.apk
releaseNotes: ${ github.event.head_commit.message || 'Manual release' }
```

8. Run the Pipeline

- **Automatic:** push to your main branch:

```
git add -A
git commit -m "ci: set up app distribution"
git push
```

- **Manual:** GitHub → **Actions** → *(your workflow)* → **Run workflow**.

Watch progress under the repository's **Actions** tab.

9. Verify Build Release

1. Firebase Console → **Run** → **App Distribution** → **Releases**.
2. Confirm the new release appears.

Firebase identifies a release by its version (`versionCode` / `versionName`). Using the workflow **run number** as the `versionCode` makes every push a new, distinct release instead of overwriting the previous one.

10. Test Build Release

1. Testers in your group receive an **email invitation** automatically.
 2. Or open the release in the console → **copy the invite link** and share it.
 3. Add more testers per release under **Add testers or group**.
-

11. Mobile Application Installation Guide (Android)

1. Open the **invitation email** from Firebase App Distribution and tap **Get started** (or open the invite link).
2. Sign in with the **Google account** that was invited.
3. If prompted, install the **Firebase App Tester** app (it manages downloads and update notifications).
4. Open your app in App Tester → tap **Download**, then **Install**.

5. If the install is blocked, allow “**Install unknown apps**” for the browser / App Tester, then retry.
6. Launch the app. Future releases appear in App Tester for one-tap updates.

Troubleshooting

Symptom (CI log)	Cause & fix
<code>./gradlew: Permission denied (exit 126)</code>	<code>gradlew</code> lacks the exec bit (common when committed from Windows). Run once: <code>git update-index --chmod=+x android/gradlew</code> (the workflow also <code>chmod +x</code> 's it).
<code>error: invalid source release: <N></code>	The runner's JDK is older than your app targets. Set <code>java-version</code> in <code>setup-java</code> to match.
Upload step: <code>PERMISSION_DENIED</code>	Service account missing Firebase App Distribution Admin — add it in IAM.
A step fails referencing a secret	A secret is missing or misnamed — add/rename it and re-run.
Corrupt keystore / base64 rejected	Re-encode without a BOM/newlines (use the clipboard method) and re-paste the secret.
<code>google-services.json</code> not found	Ensure the “Restore google-services.json” step runs before the Gradle build and the path is correct.

Appendix — Secrets Reference

Secret	Sensitivity	Notes
<code>ANDROID_KEYSTORE_BASE64</code> , <code>ANDROID_KEYSTORE_PASSWORD</code> , <code>ANDROID_KEY_PASSWORD</code>	High	Release-signing material — never commit; back up the <code>.jks</code> offline
<code>FIREBASE_SERVICE_ACCOUNT</code>	High	Grants project admin access — rotate immediately if leaked
<code>ANDROID_KEY_ALIAS</code>	Low	Your key alias
<code>FIREBASE_ANDROID_APP_ID</code>	None	Public identifier
<code>GOOGLE_SERVICES_JSON_BASE64</code>	Low	Client config; injected so it can stay out of a public repo

Tip: keep the `.jks` keystore and its passwords in a password manager. Losing the signing key means you cannot update the published app and must distribute a new one.